

AAWB: COMMERCIAL INTELLIGENCE SYSTEM

The Anam Hotel Group
Agentic AI Build Week - 24h MVP Sprint

Generated: June 30, 2026

Team Size	5 people
Timeline	24 hours
Competition	Agentic AI Build Week
Deployment	Vercel + Railway
Tech Stack	Python, React, Claude API
Demo Format	Live link + backup video

EXECUTIVE SUMMARY

The Anam hotel group spends ~2 hours every morning assembling daily reservation and performance data across disconnected systems and Excel spreadsheets before making rate, inventory, and channel decisions. Our solution: **AI agents that automatically ingest daily data, forecast demand, analyze performance against budget, detect risks, and recommend specific commercial actions** — all in under 2 minutes.

Problem: Manual, slow, inconsistent forecasting delays commercial decisions and misses market opportunities.
Solution: Multi-agent system with Claude AI for autonomous decision-making.
Expected Impact: +\$12K/month revenue from better rate timing + faster response to booking trends.

PROBLEM STATEMENT

Current State (Manual Process):

- Commercial analyst manually pulls data from 3 property management systems daily
- Data exported to Excel spreadsheets with no dedicated forecasting tool
- Compiles occupancy, ADR, revenue, booking pace against budget and last year's figures
- Inconsistent analysis across 3 properties (depends on analyst judgment)
- Insight arrives late — pace shifts and cancellation spikes often noticed too late to act

The Anam Properties:

- Property A: Urban Luxury Hotel (Hanoi, ~150 rooms)
- Property B: Beachfront Resort (Da Nang, ~200 rooms)
- Property C: Business Hotel (HCMC, ~100 rooms)

Daily Data Flow (Today):

PMS → Manual Export → Excel → Analyst Work (2 hours) → Intuition-based decisions

Desired Data Flow (After Solution):

PMS → Automated Export → AI Agents (2 minutes) → Data-driven recommendations

Data Required (Daily):

Occupancy	RevPAR	Bookings
ADR	Revenue	Cancellations
Booking Pace	Market Segment	Lead Time
Source/Channel	Guest Nationality	Performance vs Budget
Comparison vs Last Year	Forward Book (7-14 days)	Cancellation Rate

AGENT SYSTEM ARCHITECTURE

The system uses **5 specialized agents** coordinated by an **orchestrator** that manages workflow, maintains context, and ensures data flows correctly between agents. Each agent has specific tools and responsibility.

Agent	Responsibility	Tools
DATA_VALIDATOR	Validate input, check for issues	validate_schema, check_outliers, compute_metrics
FORECAST	Predict 14-day demand by segment	load_history, run_forecast_model, detect_trend
ANALYSIS	Compare vs budget, last year	compute_metrics, compare_vs_budget, variance
RISK_DETECTION	Flag anomalies & threats	analyze_cancellations, detect_pace_anomaly, risk_score
STRATEGY	Recommend actions (rate, inventory)	generate_rate_rec, optimize_inventory, channel_strategy

Workflow:

- 1. **Upload** → Daily data (CSV/JSON) uploaded to backend
- 2. **Validate** → DATA_VALIDATOR checks completeness, flags issues
- 3. **Forecast** → FORECAST_AGENT predicts 14-day demand + trends
- 4. **Analyze** → ANALYSIS_AGENT compares to budget/last year
- 5. **Detect Risks** → RISK_DETECTION_AGENT flags anomalies
- 6. **Recommend** → STRATEGY_AGENT decides rate/inventory actions (Claude AI)
- 7. **Report** → ORCHESTRATOR generates executive report with rationale

Output: 1-page daily brief with forecast, performance, risks, and specific recommendations (e.g., "Increase standard rate 8% Mon-Thu").

TECH STACK

Layer	Technology	Why
Backend	Python 3.9+ + Flask	Fast prototyping, data processing (pandas)
	Anthropic Claude API	AI agent intelligence & recommendations
	scikit-learn / Prophet	Simple, effective demand forecasting
	Pandas	Data transformation & analysis
Frontend	React 18 + Tailwind CSS	Clean UI, fast iteration, responsive
	Axios	HTTP client for backend API calls
Deployment	Vercel (Frontend)	Free tier, instant deploys, fast CDN
	Railway (Backend)	Free tier, Python support, easy secrets management
Data	Mock hotel dataset	Testing (could provide real data later)
Orchestration	Claude API (agentic)	Manages agent flow, generates final report

DEPLOYMENT STRATEGY

Primary: Live Deployed Link (Best for Judging)

- Frontend: Deployed to Vercel (~5 min setup)
- Backend: Deployed to Railway (~10 min setup)
- Judges click link → instant demo, no WiFi dependency

Backup: Localhost Demo (In-person)

- If presenting live on laptop, run both servers locally
- Pre-test all dependencies 24h before

Video Fallback: Recorded Demo (2-3 min)

- Record full workflow if deployment fails
- Upload to YouTube (unlisted) or Loom
- Shows agents running autonomously

Deployment Timeline:

- **Friday (48h before):** Deploy backend to Railway + frontend to Vercel
- **Saturday (24h before):** Record backup video
- **Demo Day:** Click live link OR show laptop OR play video

TEAM STRUCTURE & TASK BREAKDOWN

Team: 5 People | Timeline: 24 Hours

Person A	Agent Architect & LLM Lead
Role	Design agent system, write Claude prompts, Claude API integration
Tasks	• Design 5-agent system (what each does, tools, prompts) • Write system prompts for each agent • Test Claude API calls • Build orchestrator logic • Generate final report template
Deliverables	agents.py, orchestrator.py with working agent flows
Person B	Data & ML Engineer
Role	Data pipeline, forecasting model, metrics computation
Tasks	• Build mock hotel dataset generator • Implement demand forecast model (scikit-learn/Prophet) • Create data validation logic • Compute performance metrics (occupancy, ADR, RevPAR) • Implement anomaly detection
Deliverables	data_loader.py, forecast_model.py, metrics.py
Person C	Backend API Engineer
Role	Flask API, data processing pipeline, deployment
Tasks	• Build Flask API endpoints (/analyze, /health) • Connect agent orchestrator to data pipeline • Handle errors, logging, response formatting • Test end-to-end flow locally • Prepare for Railway deployment
Deliverables	api.py, requirements.txt, Railway config
Person D	Frontend Developer
Role	React UI, user flow, data visualization

Tasks	• Build React components (upload form, dashboard, results) • Create data upload interface (CSV/JSON) • Display forecast, analysis, recommendations • Show agent reasoning/logs (transparency for judges) • Make UI professional but not over-designed
Deliverables	React app, .env setup, Vercel deployment
Person E	Demo & Integration Lead
Role	End-to-end testing, demo prep, video recording, integration
Tasks	• Test full flow (upload → agents run → report) • Time the pipeline (should be <2 min) • Coordinate team integration • Practice demo script • Record 60-second backup video • Prepare Devpost submission
Deliverables	Video demo, Devpost submission, demo script

DATA STRATEGY

Data Source Approach:

The competition may or may not provide a dataset. Our strategy handles both cases.

Scenario A: Customer Provides Data

- Check Devpost + Builder Portal for dataset
- Map provided data to our schema (occupancy, ADR, revenue, bookings, etc.)
- Update data_loader.py to read provided format
- No code changes needed — agents work the same

Scenario B: No Dataset Provided (Use Mock Data)

- Generate realistic 3-property hotel data (sample_data_generator.py)
- Include realistic patterns: seasonality, booking pace, cancellations
- Clearly label demo data as "mock" in UI
- Explain in pitch: "We're using realistic mock data that mirrors actual hotel operations"

Sample Data Structure (JSON):

```
{ "date": "2024-01-15", "data_source": "mock", "properties": { "urban_luxury": { "occupancy": 0.78, "adr": 171.23, "revenue": 20053.91, "bookings_today": 5, "cancellations_today": 1 }, "forward_book": { "7_days": { "occupancy_forecast": 0.81, "pace_vs_last_year": 1.12 }, "14_days": { "occupancy_forecast": 0.78, "pace_vs_last_year": 0.98 } }, "budget": { "occupancy_target": 0.80, "adr_target": 135, "last_year": { "occupancy": 0.72, "adr": 128 } }
```

24-HOUR SPRINT TIMELINE

Time	Phase	Team Activity
Hour 0-2	SETUP & ARCHITECTURE	• A: Finalize agent design, write system prompts • B: Create mock data generator • C: Bootstrap Flask app • D: Setup React boilerplate • E: Define success criteria
Hour 2-6	AGENT DEVELOPMENT	• A: Implement agents.py, test Claude API calls • B: Implement forecast model + validation logic • C: Build API endpoints, connect to agents • D: Build upload form + mock dashboard • E: Test locally, coordinate
Hour 6-12	INTEGRATION & TESTING	• A: Finalize orchestrator logic • B: Tune forecast model accuracy • C: Full end-to-end API testing • D: Connect frontend to backend API • E: Time the pipeline (<2 min goal)
Hour 12-18	POLISH & DEPLOYMENT	• A: Refine agent prompts based on test results • B: Add edge case handling • C: Deploy to Railway + test live • D: Deploy to Vercel + test live • E: Test deployed links work
Hour 18-22	DEMO & VIDEO	• A: Review system performance • B: Generate final metrics • C: Optimize API response time • D: Make UI polished (not pretty, but professional) • E: Record 60-second backup video
Hour 22-24	FINAL PREP	• A: Practice demo script • B: Final data validation • C: Double-check deployed links • D: Verify all features work • E: Complete Devpost submission

DEMO STRATEGY FOR JUDGES

Demo Flow (2-3 minutes):

Minute 0-30 seconds: SETUP

"The Anam hotel group spends 2 hours every morning assembling data from Excel to make rate and inventory decisions. Our AI agents do this automatically."

Minute 0:30-1:00: SHOW THE PROBLEM

"Commercial team has to compare:

- Today vs budget
- Today vs last year
- Forward booking trends
- Risk signals (cancellations, pace drops)

All manually. We automate this."

Minute 1:00-1:30: RUN THE DEMO

[Upload CSV or click "Use Sample Data"]

"Watch the agents work..."

- DATA_VALIDATOR: "Validated 3 properties, 450 reservations"
- FORECAST: "Demand forecast: 78-82% occupancy, pace up 12%"
- ANALYSIS: "Today occupancy 78% vs 80% target, revenue up 8%"
- RISK: "Cancellations 8% - monitor closely Sunday"
- STRATEGY: "Increase rate 8% Mon-Thu, hold Fri-Sat. Expected +\$8K revenue"

Minute 1:30-2:00: IMPACT

"Instead of 2 hours manual work, the team has:

- ✓ Consistent forecast across all properties
- ✓ Data-driven rate recommendations
- ✓ Early risk detection (cancellations, pace changes)
- ✓ Time to act instead of time to compile

Expected impact: +\$12K/month from better rate decisions + faster response to market shifts."

[SHOW: Final report with specific recommendations]

"Questions?"

KEY SUCCESS FACTORS

■ For Judging:

- **Live Link Works:** Agents run reliably on deployed backend + frontend
- **Transparent AI Reasoning:** Judges see WHY each agent decided what (not just outputs)
- **Professional UI:** Clean, readable results (not fancy, but clear)
- **Speed:** Full analysis in <2 minutes shows efficiency
- **Real-world Relevance:** Hotel revenue management is actual business problem
- **Data Honesty:** We explain mock vs real data clearly

■ Avoid:

- ■ Demo that depends on "good WiFi" (use deployed links)
- ■ Agents that just return data (make actual decisions)
- ■ Over-engineered UI (focus on agent intelligence)

- ■ No backup plan (have video ready)
- ■ Vague recommendations ("increase rate") — be specific

■ **Winning Narrative:**

"We built a **multi-agent system** that uses **Claude AI** to autonomously analyze hotel data and recommend commercial actions. Each agent is specialized: forecast, analysis, risk detection, strategy. They work together to turn data into decisions."

PROJECT STRUCTURE & QUICK START

Repository Structure (GitHub):

anam-commercial-intelligence/ ■■■■ backend/ ■■■■ api.py # Flask app + endpoints ■■■■ main.py #
Orchestrator + agent runner ■■■■ agents.py # Agent definitions & prompts ■■■■ data_loader.py # Data
loading (real/mock/public) ■■■■ forecast_model.py # Demand forecasting logic ■■■■ metrics.py #
Performance metrics computation ■■■■ requirements.txt # Python dependencies ■■■■ .env.example #
Template for API keys ■■■■ railway.toml # Railway deployment config ■■■■ frontend/ ■■■■ src/ ■■
■■■■ App.jsx # Main React component ■■■■ components/ ■■■■ Upload.jsx # CSV upload form ■■
■■■■ Dashboard.jsx # Results display ■■■■ Report.jsx # Final recommendations ■■■■ config.js #
API endpoint config ■■■■ index.css # Tailwind CSS ■■■■ .env.production # Deployed API URL ■■■■
vercel.json # Vercel config ■■■■ data/ ■■■■ sample_data.json # Example input ■■■■ mock_generator.py
Generate realistic test data ■■■■ README.md # Setup instructions **Quick Start (Local Development):**
1. Clone & setup git clone cd anam-commercial-intelligence # 2. Backend cd backend python -m venv venv
source venv/bin/activate # or venv\Scripts\activate on Windows pip install -r requirements.txt echo
"ANTHROPIC_API_KEY=sk-..." > .env python api.py # Runs on http://localhost:5000 # 3. Frontend (new
terminal) cd frontend npm install REACT_APP_API_URL=http://localhost:5000 npm start # Runs on
http://localhost:3000 # 4. Test curl -X POST http://localhost:5000/analyze \ -H "Content-Type: application/json" \
-d @data/sample_data.json

DEPLOYMENT & FINAL CHECKLIST

48 HOURS BEFORE JUDGING:

1. Backend Deployment (Railway) — 10 min

- Push code to GitHub
- Create Railway account
- Connect GitHub repo
- Add ANTHROPIC_API_KEY as secret
- Deploy → Get live URL (e.g., <https://anam-prod.railway.app>)

2. Frontend Deployment (Vercel) — 5 min

- Update REACT_APP_API_URL to live backend URL
- Push to GitHub
- Create Vercel account
- Connect GitHub repo → auto-deploys
- Get live URL (e.g., <https://anam-demo.vercel.app>)

3. Testing

- Test live frontend link (can you load it?)
- Upload sample data → does backend run?
- Do agents produce output in <2 min?
- Does final report display correctly?
- Test on mobile (responsive?)
- Test on different WiFi network

4. Video Backup

- Record 60-second demo with Loom or OBS
- Show: upload → agents running → recommendations
- Upload to YouTube (unlisted)
- Get shareable link

5. Devpost Submission

- Write project description
- List tech stack
- Explain data approach (mock vs real)
- Include live link
- Include backup video link
- Add team member names/roles

JUDGING DAY:

- Have laptop with demos ready
- Have phone hotspot as WiFi backup
- Practice 2-min demo script (no script needed, know the flow)
- Know why each agent matters
- Be ready to explain any design choice

ESSENTIAL CONTACTS & RESOURCES:

- **Claude API:** <https://console.anthropic.com> (get API key here)
- **Railway:** <https://railway.app> (backend deployment)
- **Vercel:** <https://vercel.com> (frontend deployment)
- **Devpost:** Check project description for submission link
- **Build Instructions:** Check AAWB Builder Portal for data + specific requirements

Quick Links:

- Flask Docs: <https://flask.palletsprojects.com>
- React Docs: <https://react.dev>
- Claude API Docs: <https://docs.anthropic.com>
- Tailwind CSS: <https://tailwindcss.com>